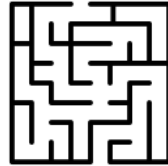


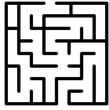
PI de Sistemas de Informação Distribuídos (25/26)

Licenciaturas em Engenharia Informática e Informática e Gestão de Empresas



<u>Grupo</u>	<u>6</u>
122663 José Martins de Moura Gonçalves Jarmela jmmgj@iscte-iul.pt	
122666 Nuno Tiago Teixeira Neves nttns@iscte-iul.pt	
122678 Gonçalo Costa Rua gcraa@iscte-iul.pt	
122693 Tiago Filipe Soares Horta Nogueira tfshn@iscte-iul.pt	
122703 Eduardo David Rivas Correia edrca@iscte-iul.pt	
123854 Pedro Branco de Brito Vaz Almeida pbbva@iscte-iul.pt	

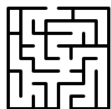
Grupo com quem trocam especificações: 5



Instruções

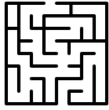
Estas instruções são de cumprimento obrigatório. Relatórios que não cumpram as indicações serão penalizados na nota final.

- Podem (e em várias situações será necessário) ser adicionadas novas páginas ao relatório, mas não podem ser removidas páginas. Se uma secção não for relevante, fica em branco, não pode ser removida;
- A paginação tem de ser sequencial e não ter falhas;
- O índice tem de estar atualizado.
- O grupo que inicia o documento (coluna à esquerda na folha de rosto) preenche apenas a parte inicial (até ao fim da secção 1). Este documento word vai ser colocado no moodle para que o outro grupo (à direita da folha de rosto) possa descarregar e continuar a preenchê-lo (secção 2)



Índice

1	Especificação	5
1.1	Da Nuvem para o Mongo	5
1.2	Descrição Geral do Procedimento de Mongo Para Mysql	8
1.3	Tratamento de dados anómalos (valores de sensores errados)	12
1.4	Tratamento de outliers de ruído	13
1.5	Tratamento de Alertas de ruído e temperatura	14
1.6	Tratamento de número de marsamis numa sala (obter pontuação)	16
1.7	Especificação de Store Procedures SQL de apoio à migração e tratamento de dados 17	
1.8	Especificação de Triggers de apoio à migração e tratamento de dados	18
1.9	Modelo Relacional	19
1.10	Utilizadores Base de Dados Mysql	21
1.11	Procedimentos Manutenção da Aplicação	22
1.12	Eventos de suporte à aplicação (caso existam)	23
1.13	Consulta por HTML/PHP	24
2	Implementação	26
2.1	Coleções a criar em cada uma das réplicas do Mongo	26
2.2	Descrição Geral do Procedimento de Mongo Para Mysql	27
2.3	Tratamento de dados anómalos (valores de sensores errados)	29
2.4	Tratamento de outliers de ruído	30
2.5	Tratamento de outliers de temperatura	31
2.6	Tratamento de Alertas de Som	32
2.7	Tratamento de Alertas de Temperatura	33
2.8	Tratamento de número de marsamis numa sala (obter pontuação)	34
2.9	Implementação de Stored Procedures SQL de apoio à migração e tratamento de dados	35
2.10	Implementação de Triggers	36
2.11	Modelo Relacional	37
2.12	Utilizadores Base de Dados Mysql	38
2.13	Procedimentos Manutenção da Aplicação	39



PI de Sistemas de Informação Distribuídos, 25/26

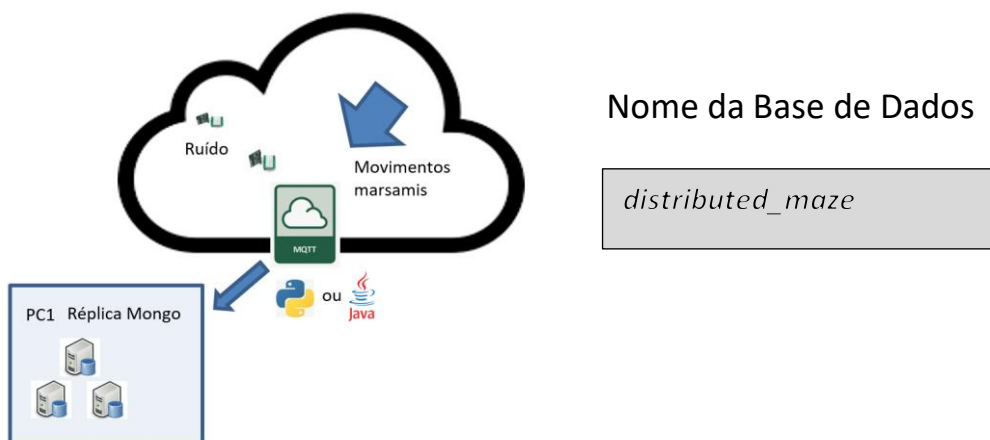
2.14	Eventos de suporte à aplicação (caso existam)	40
2.15	PrintScreen dos formulários HTML implementados	41
2.16	PrintScreen dos formulários Android com dados	42
	Código de Triggers implementados	44
	Código Stored Procedures implementados	45



1 Especificação

Esta secção é onde o grupo que inicia o documento (coluna à esquerda na folha de rosto) coloca a especificação do que pretende implementar. Mais tarde pode implementar de outra maneira, mas aqui vão as primeiras ideias que serão avaliadas na primeira oral e que vão ser entregues a outro grupo para que analisem e vejam se aproveitam as vossas ideias.

1.1 Da Nuvem para o Mongo



Nome Coleção	O que armazena?
temperature_events	Regista a temperatura no labirinto, num determinado momento. (seq, player, timestamp, value, migrated)
sound_events	Regista o ruído no labirinto, num determinado momento. (seq, player, timestamp, value, migrated)
movement_events	Regista o movimento de um marsami. (seq, player, marsami_id, origin, destiny, status, timestamp, migrated)
room_occupancy	Regista a ocupação atual de cada sala do labirinto, incluindo o número de marsamis odd e even presentes na sala. (seq, room_id, odd, even, total, timestamp, migrated)
system_events	Regista os diferentes eventos do sistema, incluindo alertas de sensores e eventos de jogo. (Hora, Sensor, Leitura, TipoEvento, Msg, Seq, migrated)



counters	Guarda contador atómico global utilizado para gerar números de sequência únicos e ordenados. (seq)
-----------------	--

Coleção: **temperature_events**

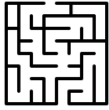
```
{
  "_id": {
    "$oid": "69b1da7d72a054eb41feAAF7"
  },
  "seq": 101,
  "player": 6,
  "timestamp": {
    "$date": "2026-03-11T21:11:25.736Z"
  },
  "value": 15.609060150000001,
  "migrated": true
}
```

Coleção: **sound_events**

```
{
  "_id": {
    "$oid": "69b1da8f72a054eb41feab1c"
  },
  "seq": 48,
  "player": 6,
  "timestamp": {
    "$date": "2026-03-11T21:11:43.831Z"
  },
  "value": 10.824321600000001,
  "migrated": true
}
```

Coleção: **movement_events**

```
{
  "_id": {
    "$oid": "69b1da8b72a054eb41feab14"
  },
  "seq": 113,
  "player": 6,
  "marsami_id": 9,
}
```



```
"origin": 6,  
"destiny": 8,  
"status": 1,  
  
"timestamp": {  
  "$date": "2026-03-11T21:11:39.609Z"  
},  
"migrated": true  
}
```

Coleção: **room_occupancy**

```
{  
  "_id": {  
    "$oid": "69b1da5af6b2b0fc38acd85a"  
  },  
  "seq": 131,  
  "room_id": 1,  
  "even": 1,  
  "odd": 2,  
  "timestamp": {  
    "$date": "2026-03-11T21:12:47.741Z"  
  },  
  "total": 3,  
  "migrated": true  
}
```

Coleção: **system_events**

```
{  
  "_id": {  
    "$oid": "69bb013fb61fab752d84f01a"  
  },  
  "seq": 128,  
  "player": 6,  
  "timestamp": {  
    "$date": "2026-03-18T19:47:11.958Z"  
  },  
  "sala": null,  
  "sensor": "som",  
  "leitura": 27.454197896550774,  
  "tipo": "RuidoAlto",  
  "msg": "Alerta: Ruído 27.45 acima do limite 27.00 (nível  
1) ",  
}
```



```
"migrated": true  
}
```

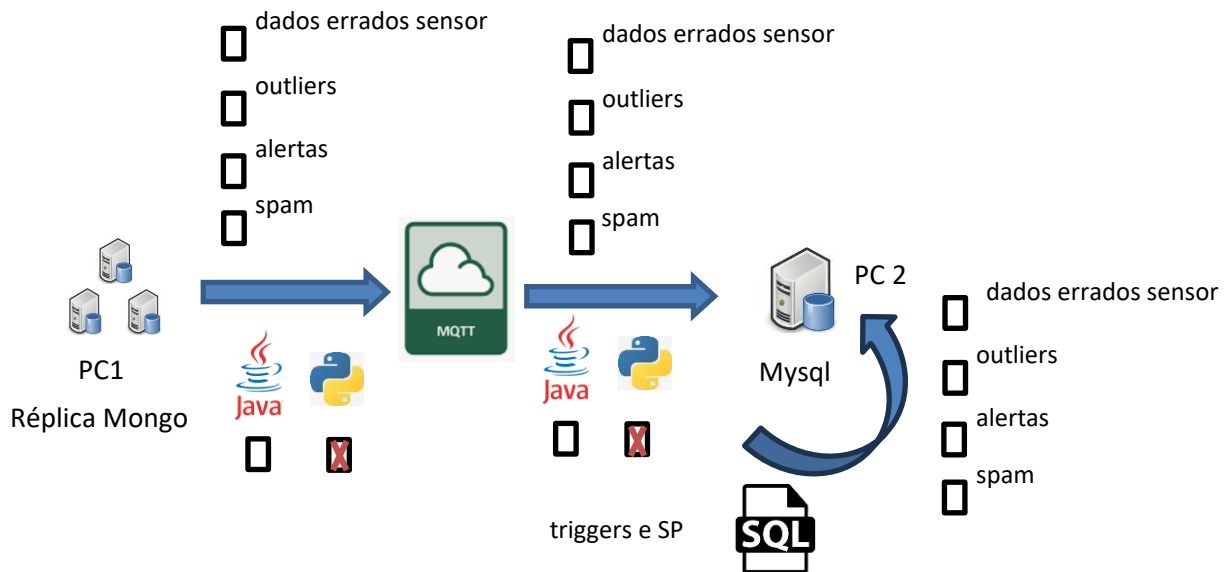
Coleção: **counters**

```
{  
  "_id": "seq",  
  "seq": 284  
}
```



1.2 Descrição Geral do Procedimento de Mongo Para Mysql

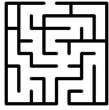
A passagem do Mongo para Mysql tem duas fases: a) enviar de Mongo para MQTT e b) receber de MQTT para Mysql. No diagrama deve ser indicado (nas check box) em qual das fases são programadas (em java e/ou python) as tarefas enumeradas (podem ser na fase a) b) ou ambas).



- Com que “periodicidade” (segundos) o programa vai buscar ao mongo: 2
- Como garantem que não enviam duas vezes o mesmo documento do Mongo para o Mysql / MQTT (com base em datas ou booleano ou etc.):

Cada documento no MongoDB possui um campo booleano migrated, inicialmente definido como false. O componente responsável pela migração (MigrationWorker) seleciona apenas documentos com migrated = false, garantindo que apenas dados ainda não processados são enviados para o MQTT.

Após a publicação, o documento não é imediatamente marcado como migrado. Em vez disso, aguarda-se uma confirmação explícita, enviada pelo componente responsável pela escrita no MySQL (MySQLWriter) através de um tópico MQTT de confirmação.



Só após essa confirmação o campo `migrated` é atualizado para `true`. Em caso de falha no MySQL, essa confirmação não é enviada, mantendo o documento com `migrated = false`, o que garante o seu reenvio numa iteração futura.

Desta forma, assegura-se que nenhum documento é perdido e evita-se a duplicação de processamento ao nível da aplicação.

- Pensam usar threads do Mongo para MQTT?: Não e do MQTT para Mysql? Não
- Quantas threads e/ou quantos mains em cada um dos passos?: 1 main loop por componente
- No transporte MQTT que QoS vão utilizar? Porquê?

No transporte MQTT foi utilizado QoS 1 (At Least Once). Este nível de qualidade de serviço garante que cada mensagem é entregue pelo menos uma vez ao destinatário. Caso ocorra alguma falha na comunicação, o broker pode reenviar a mensagem até receber confirmação de que foi entregue.

O QoS 0 não foi utilizado por não garantir a entrega de mensagens. O QoS 2 também não foi escolhido, pois implica um protocolo de confirmação mais complexo (vários passos de handshake), aumentando o overhead e a latência.

A eventual duplicação de mensagens resultante do uso de QoS 1 é tratada ao nível da base de dados. Cada evento é identificado por um número de sequência (`seq`) e associado ao identificador da simulação (`IDJogo`), existindo uma restrição de unicidade sobre este par de atributos. Desta forma, tentativas de inserção duplicada são rejeitadas ou ignoradas, garantindo que cada evento é processado apenas uma vez.



Aqui podem desenvolver informação que considerem relevante relativo ao processo de migração, aspectos que não esteja refletido nas secções seguintes.

Para além do processo de migração, o sistema segue uma arquitetura organizada em duas fases distintas: processamento de dados e persistência.

Numa primeira fase, após a receção das mensagens via MQTT, os dados são processados em tempo real no backend intermédio. Este processamento inclui validação das mensagens, deteção de outliers, geração de alertas e execução de ações de controlo do sistema. Apenas dados considerados válidos e consistentes seguem para as etapas seguintes.

Numa segunda fase, os dados já processados são armazenados no MongoDB, que funciona como sistema de persistência intermédio. Desta forma, evita-se o armazenamento de dados inválidos ou irrelevantes, garantindo que apenas informação fiável é mantida.

Posteriormente, ocorre o processo de migração entre MongoDB e MySQL, realizado em duas fases:

(a) leitura de documentos no MongoDB e publicação no MQTT.

(b) receção das mensagens via MQTT e inserção no MySQL.

Os componentes de migração são independentes, permitindo que falhas num dos componentes não bloqueiem os restantes. Cada componente utiliza um ciclo principal sequencial (main loop), responsável por executar as tarefas de forma ordenada, evitando problemas de concorrência e simplificando a implementação.

A comunicação com o broker MQTT não bloqueia o ciclo principal, permitindo que o sistema continue a processar dados enquanto as mensagens são enviadas e recebidas.

Cada mensagem armazenada no MongoDB é associada a um número de sequência (seq), gerado no backend intermédio através de um contador mantido na coleção counters. A obtenção do próximo valor é realizada de forma atómica, recorrendo a uma operação de incremento, garantindo que não existem duplicações.



Cada documento possui ainda o campo `migrated`, inicialmente definido como `false`. O componente de migração (`MigrationWorker`), executado a cada 2 segundos, seleciona apenas documentos com `migrated = false`, ordenados por `seq`, e publica-os no tópico `pisid_migrate_all` no MQTT. Este intervalo foi definido tendo em conta que o sistema recebe dados com uma frequência de aproximadamente 1 segundo, permitindo agrupar múltiplas mensagens por ciclo, reduzir overhead de comunicação e manter uma latência baixa e controlada na migração.

Após a publicação, o documento não é imediatamente marcado como migrado. O sistema aguarda uma confirmação explícita do componente responsável pela escrita no MySQL (`MySQLWriter`), enviada através do tópico `pisid_migrate_confirm`.

O componente responsável pela escrita no MySQL subscreve o tópico de migração, insere os dados na base de dados e publica uma confirmação:

- `success = true` -> documento marcado como `migrated = true`
- `success = false` -> documento mantém-se não migrado e será reenviado

Para garantir a ordem de inserção, é utilizado um buffer ordenado por `seq`, processando apenas mensagens quando o valor esperado está disponível.

Em caso de falha no MySQL, a confirmação não é bem-sucedida, o documento mantém-se com `migrated = false` e o sistema tenta reconectar, garantindo o reenvio automático e evitando perda de dados.

Esta abordagem segue o princípio "at least once", garantindo que nenhuma mensagem é perdida, mesmo que possam ocorrer duplicações, tratadas ao nível da base de dados.



1.3 Tratamento de dados anómalos (valores de sensores errados)

Aqui devem explicar o que fazer caso se detetem valores “errados” (datas impossíveis, caracteres estranhos, etc..). Se recorrerem a triggers ou SP então indicam em secção mais adiante.

O sistema recebe dados provenientes de sensores através do protocolo MQTT, incluindo leituras de temperatura, som e eventos de movimento de marsamis entre salas.

A deteção de valores errados é realizada no backend intermédio, imediatamente após a receção das mensagens MQTT e antes do armazenamento dos dados no MongoDB. Esta etapa corresponde à primeira fase do processamento, garantindo que apenas dados validados entram no sistema.

Durante esta fase são verificadas as seguintes condições:

Validação estrutural da mensagem - confirmação de que a mensagem recebida possui todos os campos obrigatórios (por exemplo identificador de marsami, sala de origem e destino ou valor do sensor).

Validação de tipos de dados - verificação de que os campos possuem o tipo esperado (por exemplo valores numéricos para leituras de sensores ou identificadores inteiros).

Validação de timestamps - verificação de que a data e hora são válidas e coerentes.

Normalização de campos - tratamento de possíveis erros de formatação (por exemplo caracteres inesperados ou variações nos nomes dos atributos), garantindo consistência antes do processamento.

Se alguma verificação falhar, a mensagem é descartada e não segue para processamento posterior. Caso contrário, após passar em todas as verificações com sucesso, a mensagem é considerada válida e continua no pipeline.

Apenas mensagens válidas são utilizadas para atualizar o estado interno do sistema, bem como para a geração de alertas e execução de ações automáticas. Estas são posteriormente armazenadas no MongoDB e, através do processo de migração, inseridas na base de dados MySQL.



1.4 Tratamento de outliers de ruído

Aqui devem explicar o que fazer caso se detetem “outliers” (valores fisicamente possíveis mas irrealistas, como por exemplo variações muito bruscas do ruído apenas num segundo). Se recorrerem a triggers ou SP então indicam em secção mais adiante.

As leituras de ruído podem apresentar outliers, ou seja, valores fisicamente possíveis, mas pouco plausíveis face ao comportamento do sistema (por exemplo, variações muito abruptas num curto intervalo de tempo).

Para a sua deteção, o sistema utiliza a média acumulada dos valores considerados válidos ao longo da simulação. Para cada nova leitura, calcula-se a média e compara-se com o valor atual. Se a diferença absoluta ultrapassar um limiar definido pelo utilizador, a leitura é classificada como outlier.

Formalmente, uma leitura é considerada outlier quando:

$$|\text{valor} - \text{média}| > \text{limiar}$$

Quando uma leitura é identificada como outlier, esta é descartada e não é utilizada para atualizar a média nem para qualquer processamento posterior, nomeadamente geração de alertas ou armazenamento em base de dados.

Esta abordagem assume que o comportamento esperado do sistema é relativamente estável ao longo da simulação, sendo adequada para detetar picos isolados de valores anómalos. Alterações graduais e consistentes ao longo do tempo não são consideradas outliers, uma vez que a média é atualizada apenas com valores válidos.

Desta forma, assegura-se que leituras inconsistentes não afetam o processamento nem a geração de alertas, garantindo que apenas dados fiáveis entram no sistema.



1.5 Tratamento de Alertas de ruído e temperatura

Aqui devem explicar o que fazer caso se detetem situações alarmantes relativos ao som e temperatura. Se recorrerem a triggers ou SP então indicam em secção mais adiante. Que situações despoletam os alertas, onde e como são armazenados. Explicar se existem mecanismos para evitar "spam" (demasiadas mensagens). É aconselhável recorrer a esquemas gráficos para explicar o mecanismo.

O sistema monitoriza continuamente os valores dos sensores de temperatura e ruído. Para cada sensor, é definido um valor limite com base nos parâmetros armazenados na tabela SetupMaze da base de dados remota.

No caso da temperatura, o limite máximo é calculado como $\text{normaltemperature} + \text{temperaturevarhightoleration}$. Para o ruído, o limite é dado por $\text{normalnoise} + \text{noisevartoleration}$.

Para detetar a aproximação a situações críticas, foram definidos três níveis de checkpoint. Cada nível corresponde a uma fração da tolerância adicionada ao valor base:

- nível 1: 25% da tolerância
- nível 2: 50% da tolerância
- nível 3: 75% da tolerância

A geração de alertas é realizada após a validação e processamento das leituras, garantindo que apenas dados válidos e filtrados são considerados antes de serem armazenados no sistema.

Para cada nova leitura, é determinado o nível atual e comparado com o nível anterior. Um alerta é gerado apenas nas seguintes situações:

- quando ocorre uma subida de nível
- ou quando o valor se mantém acima do limite e já passou o intervalo mínimo definido para o nível atual.

As leituras em descida não geram novos alertas, mesmo que permaneçam acima do limite, evitando emissões redundantes.

A frequência mínima entre alertas depende do nível:

- nível 1: a cada 10 segundos
- nível 2: a cada 7 segundos
- nível 3: a cada 5 segundos



Quando um alerta é gerado, o evento é armazenado no MongoDB na coleção `system_events`, sendo posteriormente migrado para a tabela `Mensagens` no MySQL.

Para além da geração de alertas, o sistema executa automaticamente ações de controlo com base nas leituras recebidas, sem intervenção do utilizador.

Na temperatura, quando o valor atinge o nível 1 de alerta, o sistema ativa o ar condicionado. Este mantém-se ativo enquanto a temperatura se mantiver acima do valor normal, sendo apenas desligado quando regressa a `normaltemperature`.

No ruído, quando o valor atinge o nível 1 de alerta, o sistema inicia o fecho progressivo de portas. As portas são fechadas uma a uma, a cada emissão de alerta, enquanto o ruído se mantiver acima do limite e existirem portas disponíveis para fechar. Caso o valor do ruído comece a diminuir (ainda acima do limite), o sistema interrompe o fecho de novas portas, evitando ações desnecessárias. Quando o ruído regressa a valores normais (`normalnoise`), o sistema reabre todas as portas previamente encerradas.





1.6 Tratamento de número de marsamis numa sala (obter pontuação)

Aqui devem explicar como funciona o mecanismo que deteta que o número de marsamis odd é (ou vai ser) igual ao número de marsamis even. Como deteta, e o que desencadeia.

O número de marsamis em cada sala é mantido dinamicamente no backend intermédio, responsável pelo processamento das mensagens após a sua receção via MQTT.

A cada evento de movimento, o sistema atualiza os contadores da sala de origem e de destino, distinguindo marsamis de identificador ímpar (odd) e par (even). Os eventos são processados de acordo com a ordem definida pelo campo seq, garantindo consistência mesmo em situações de chegada fora de ordem.

Para cada sala, o sistema mantém em memória o estado atual (equilíbrio ou desequilíbrio), permitindo detetar transições entre estados. Após cada atualização, é verificada a condição de equilíbrio, ou seja, se o número de marsamis odd é igual ao número de marsamis even na sala.

Para evitar múltiplos disparos consecutivos, o sistema apenas reage quando ocorre uma transição de estado, isto é, quando a sala passa de uma situação de desequilíbrio para equilíbrio.

Quando esta condição é satisfeita, é gerado um evento que é registado na coleção system_events no MongoDB, sendo posteriormente migrado para o MySQL.

Este processamento é realizado em tempo real, sem necessidade de consultas adicionais à base de dados, garantindo elevada eficiência e baixa latência na deteção da condição de pontuação. Em caso de reinício do sistema, o estado pode ser reconstruído a partir dos dados persistidos.



1.7 Especificação de Store Procedures SQL de apoio à migração e tratamento de dados

Nas secções anteriores foram descritos mecanismos que podem ou não necessitar de recorrer a Store Procedures. É nesta tabela que eles deverão ser listados. Na descrição apenas colocar informação que não seja óbvia.

Nome SP	Argumentos	Muito breve descrição
(não implementados)		



1.8 Especificação de Triggers de apoio à migração e tratamento de dados

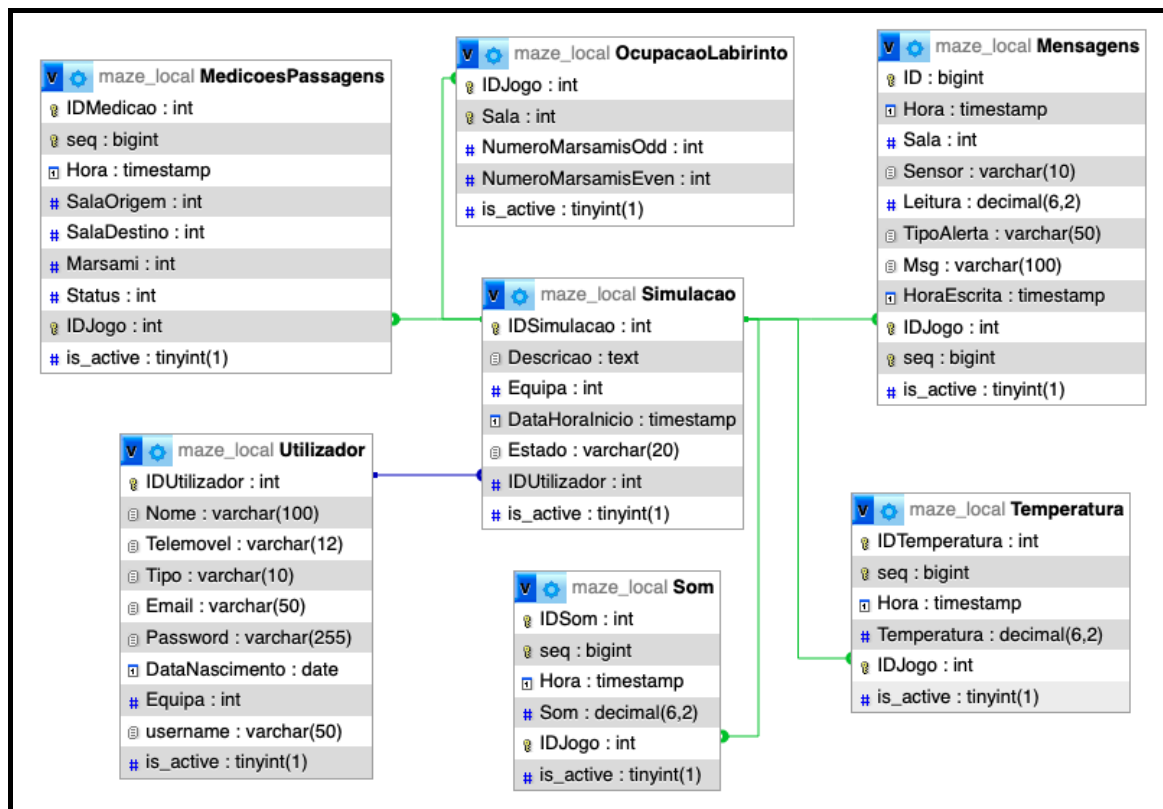
Nas secções anteriores foram descritos mecanismos que podem ou não necessitar de recorrer a Triggers. É nesta tabela que eles deverão ser listados. Nas Notas apenas colocar informação que não seja óbvia.

Nome Trigger	Tabela	Tipo de Operação (I, U, D)	Evento (After, Before)	Notas
(não implementados)				



1.9 Modelo Relacional

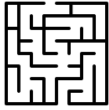
Diagrama relacional completo (modo gráfico). Alterações à base de dados original terão de ser justificadas aqui. Caso seja pertinente podem ser adicionadas comentários a justificar opções pouco óbvias.



A base de dados final apresenta algumas alterações relativamente ao modelo inicial proposto, com o objetivo de melhorar a consistência dos dados, suportar o sistema de migração e a adaptar a estrutura ao funcionamento real da aplicação.

Uma das principais alterações foi a introdução do campo IDJogo nas tabelas de medições (Temperatura, Som, MedicoesPassagens, Mensagens e OcupacaoLabirinto), permitindo associar todos os dados a uma simulação específica. Esta alteração garante isolamento entre execuções, facilita consultas e análise de dados por jogo, e permite a futura extensão da aplicação, nomeadamente a visualização de dados de simulações passadas.

Foi também adicionado o campo seq (número de sequência) nas tabelas de eventos, com o objetivo de preservar a ordem global das mensagens provenientes do



sistema distribuído (MongoDB → MQTT → MySQL) e evitar duplicados.

Adicionalmente, foi introduzido o campo `is_active` em várias tabelas, permitindo implementar um mecanismo de desativação lógica (soft delete). Esta abordagem evita a remoção física de registos, preservando o histórico dos dados e garantindo maior flexibilidade na gestão da informação. A desativação é efetuada através de stored procedures, sendo aplicada de forma consistente às entidades relacionadas com uma simulação.

A tabela `Simulacao` foi também ajustada, passando a incluir o campo `IDUtilizador`, de forma a associar cada simulação ao utilizador que a criou, suportando controlo de permissões através de stored procedures.

Por fim, a tabela `OcupacaoLabirinto` foi estruturada com chave composta (`IDJogo`, `Sala`), permitindo manter o estado atualizado do número de marsamis por sala de forma eficiente e consistente.



1.10 Utilizadores Base de Dados Mysql

Nesta secção deverá ser explicado de que forma deverá ser feita a manutenção de utilizadores Mysql. Nomeadamente deverá ser indicado, para cada tipo de utilizador, que privilégios ele tem sobre que tabelas e Stored Procedures (todos os SP usados na Aplicação

Tabela	Tipo de Utilizador		
	Administrador	Utilizador (User)	Software
Utilizador	U/I/D/L	U	L
Simulacao	U/I/D/L	L	U/L
MedicoesPassagens	U/I/D/L	L	I/L
Temperatura	U/I/D/L	L	I/L
Som	U/I/D/L	L	I/L
OcupacaoLabirinto	U/I/D/L	L	I/L
Mensagens	U/I/D/L	L	I/L
Stored Procedures			
SP.CriarJogo	X	X	X
SP.EditarJogo	X	X (Apenas os seus)	X
SP.ApagarJogo	X	X (Apenas os seus)	-
SP.AdicionarUtilizador	X	-	-
SP.EditarUtilizador	X	X (Apenas os próprios dados)	-
SP.ApagarUtilizador	X	-	-

Em que U=Update, I Insert, D- Delete, L=Leitura, X=Executar SP e - sem permissões.



1.11 Procedimentos Manutenção da Aplicação

Nesta secção deverão ser listados os SP para a manutenção de utilizadores e jogos (apenas os obrigatórios)

Nome SP	Argumentos	Muito breve descrição
CriarJogo	p_equipa, p_descricao, p_dataHoraInicio , p_estado, OUT p_idJogo	Cria uma simulação associada ao utilizador autenticado
EditarJogo	p_idJogo, p_descricao, p_estado	Edita os dados de uma simulação; apenas o criador ou admin pode editar
ApagarJogo	p_idJogo	Desativa uma simulação e dados associados; apenas criador ou admin podem executar
AdicionarUtilizador	p_Nome, p_Telemovel, p_Tipo, p_Email, p_DataNascimento , p_Password, p_Username	Cria utilizador na BD e no MySQL com permissões adequadas ao tipo
EditarUtilizador	p_Nome, p_Telemovel, p_Tipo, p_Email, p_DataNascimento , p_Username	Edita dados pessoais; cada utilizador edita os seus próprios dados
ApagarUtilizador	p_username	Desativa o utilizador e remove o acesso ao MySQL; apenas admin pode executar



1.12 Eventos de suporte à aplicação (caso existam)

Deverão ser indicados todos os eventos relevantes para o processo de migração, eventos do Windows e do Mysql. Por eventos entende-se as tarefas do Windows ou eventos do Mysql

Nome Evento	Local Execução (Mysql/Window s)	Muito breve descrição



1.13 Consulta por HTML/PHP

Desenhar o layout dos formulários pretendidos, se relevante colocar texto a explicar a funcionalidade pretendida. Formulários:

- Fazer login
- Criar (ou seleccionar de uma lista de jogos um para alterar) um jogo e, para esse jogo, editar os valores associados. Quando está a alterar não pode alterar valores de chaves estrangeiras e primárias.

Indicar para cada botão qual o SP que deverá ser executado

The image shows a simple login form titled "Login". It contains two input fields: "Username" and "Password". Below the password field is a blue button labeled "Entrar". The form is centered on a light gray background.

Figura 1 – Formulário de autenticação (login)

O botão **“Entrar”** desencadeia a execução do processo de autenticação, não sendo utilizado um *stored procedure* específico, mas sim uma verificação direta das credenciais na tabela *Utilizador*.

The image shows a web application interface. On the left is a dark blue sidebar menu with the title "Menu" and items: "Dashboard", "Os seus jogos", "Criar jogo", and "Logout". The main content area has a light gray background. At the top of this area, it says "Criar Novo Jogo" on the left and "Utilizador: joao123" on the right. In the center is a white form titled "Novo jogo". The form has three input fields: "Descrição" (with a placeholder "Descrição do jogo"), "Variação máxima de Temperatura (°C)" (with a placeholder "Ex: 5"), and "Variação máxima de Som (dB)" (with a placeholder "Ex: 5"). At the bottom of the form are two buttons: a green "Criar jogo" button and a red "Cancelar" button.

Figura 2 - Formulário de criação de novo jogo



O botão **“Criar jogo”** desencadeia a execução da stored procedure **CriarJogo**, responsável por inserir a nova simulação na base de dados e associá-la ao utilizador autenticado.

O botão **“Cancelar”** permite abortar a operação sem efetuar qualquer alteração.

ID Simulacao	Descrição	Data	Estado	Ações
2	Descrição 2	05/04/2026 - 20:00	Concluído	<button>Edit</button> <button>Delete</button>

Figura 3 - Listagem dos jogos do utilizador

Para cada jogo, são disponibilizadas diferentes ações, dependendo do seu estado:

- **Editar** – permite alterar os dados da simulação através da execução do stored procedure **EditarJogo**.
- **Eliminar** – permite eliminar a simulação através de desativação lógica, marcando-a como inativa juntamente com os dados associados, através da execução da stored procedure **ApagarJogo**.

Simulação concluída — apenas o nome e a descrição podem ser alterados.

Descrição

Variação máxima de Temperatura (°C)

Ex: 5

Variação máxima de Som (dB)

Ex: 5

Cancelar Guardar

Figura 4 - Ecrã de edição de jogo

O botão **“Guardar”** executa a stored procedure **EditarJogo**, apenas permitindo editar a descrição do jogo. O botão **“Cancelar”** apenas fecha a janela.



2 Implementação

. Aqui vão falar da implementação que fizeram. A implementação é o “best of”, ou seja, o que agora acham que é a melhor solução, com base em tudo o que aprenderam (com as vossas experiências, com as ideias do outro grupo, discussões com o professor, google, chatgpt, etc), no limite podem não seguir

2.1 Coleções a criar em cada uma das réplicas do Mongo

Versão	Número Coleções
Especificação inicial	6
Recebida outro grupo	1
<u>Implementada</u>	6

Justificação da escolha

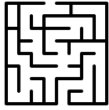
Optou-se por manter as coleções especializadas inicialmente definidas (`temperature_events`, `sound_events`, `movement_events`, `room_occupancy` e `system_events`), em vez de utilizar uma coleção genérica como `sensor_events`, por uma questão de clareza e organização.

A separação por tipo de evento torna a estrutura de dados mais explícita e facilita a manutenção, evitando a necessidade de distinguir diferentes tipos de eventos dentro da mesma coleção.

Além disso, esta abordagem simplifica o processamento dos dados, uma vez que cada tipo de evento possui atributos e regras próprias, reduzindo a complexidade e a probabilidade de inconsistências.

Coleção: Temperature events

```
{
  "_id": {
    "$oid": "69f12259908ca4adf65040d9"
  },
  "seq": 162,
  "player": 6,
  "timestamp": {
    "$date": "2026-04-28T22:10:49.701Z"
  }
}
```



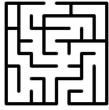
```
    },  
    "value": 60,  
    "is_valid": false,  
    "reason": "Temperature outlier",  
    "migrated": true  
}
```

Coleção: sound events

```
{  
  "_id": {  
    "$oid": "69f12256908ca4adf65040d4"  
  },  
  "seq": 157,  
  "player": 6,  
  "timestamp": null,  
  "value": 12.433743083946524,  
  "is_valid": false,  
  "reason": "Invalid timestamp",  
  "migrated": true  
}
```

Coleção: movement events

```
{  
  "_id": {  
    "$oid": "69f127a6f1622126e0dd3e85"  
  },  
  "seq": 581,  
  "player": 6,  
  "marsami_id": 19,  
  "origin": 2,  
  "destiny": 8,  
  "status": 1,  
  "is_valid": false,  
  "reason": "Origin mismatch",  
  "timestamp": {  
    "$date": "2026-04-28T22:33:26.887Z"  
  },  
  "migrated": true  
}
```



Coleção: room_occupancy

```
{
  "_id": {
    "$oid": "69f121d6b2d3239983591c86"
  },
  "seq": 2,
  "room_id": 3,
  "odd": 1,
  "even": 0,
  "total": 1,
  "timestamp": {
    "$date": "2026-04-28T22:08:38.245Z"
  },
  "migrated": true
}
```

Coleção: system_events

```
{
  "_id": {
    "$oid": "69f1225e908ca4adf65040e7"
  },
  "seq": 176,
  "player": 6,
  "timestamp": "2026-04-28T22:10:54.156140",
  "sala": null,
  "sensor": "Som",
  "leitura": 14.28246247576273,
  "tipo": "RuidoAlto",
  "msg": "Nível 1: Ruído Excessivo",
  "migrated": true
}
```

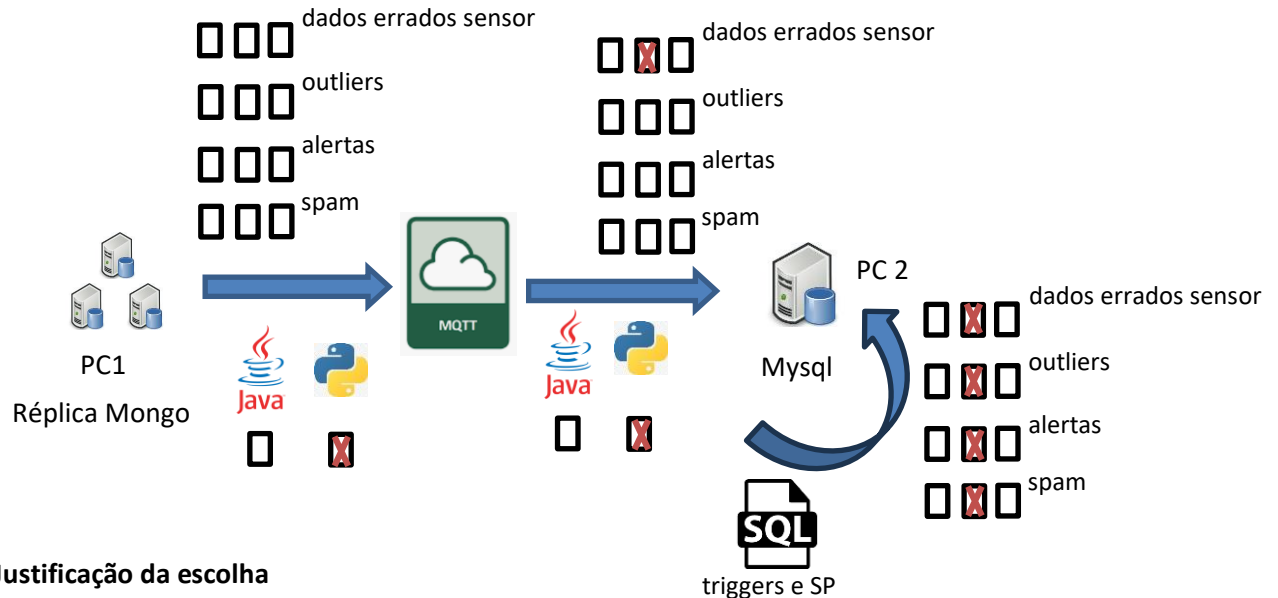
Coleção: counters

```
{
  "_id": "seq",
  "seq": 667
}
```



2.2 Descrição Geral do Procedimento de Mongo Para Mysql

Na checkbox da esquerda indicam o que especificaram inicialmente, na do meio a especificação do outro grupo e na da direita a vossa implementação.



Optou-se por realizar toda a validação e processamento dos dados antes da sua persistência no MongoDB, em vez de delegar essa responsabilidade para etapas posteriores do sistema, como no MySQL.

Esta abordagem garante que apenas dados já validados e tratados são armazenados, evitando a propagação de informação inválida ao longo do pipeline. Assim, o MongoDB passa a funcionar como uma fonte de dados consistente e fiável.

Além disso, centralizar a lógica de validação no momento da ingestão simplifica a arquitetura, reduz a dependência de triggers ou processamento adicional em MySQL e torna o comportamento do sistema mais previsível.

Versão	Periodicidade vai buscar Mongo	Como evitam enviar duas vezes para MQTT	Número Threads	QOS
Especificação inicial	2	Os documentos só são marcados como migrados após confirmação do MySQL, garantindo	0	1



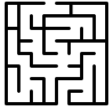
		reenvio em caso de falha. Duplicações são evitadas através de uma restrição de unicidade (seq, IDJogo).		
Recebida outro grupo	0.5	O reenvio de documentos é intencional para garantir fiabilidade, mas não há duplicação no MySQL graças a identificadores únicos e registo de eventos processados. Após confirmação, o documento é marcado e não volta a ser enviado.	1	1
<u>Implementada</u>	2	Especificação inicial	0	1

Justificação da escolha

Optou-se pela abordagem baseada no campo migrated e confirmação via MQTT por permitir um controlo direto do estado de cada documento ao longo do pipeline de migração, mantendo essa informação no próprio MongoDB e evitando estruturas externas adicionais.

Comparativamente à solução do outro grupo, que recorre a uma tabela (EventosProcessados) para garantir idempotência, a abordagem adotada centraliza a lógica no fluxo MongoDB → MQTT → MySQL, reduzindo a complexidade e o acoplamento do sistema.

O mecanismo de confirmação garante que um documento só é marcado como migrado após sucesso no MySQL, permitindo reenvio em caso de falha sem perda de dados. A eventual duplicação ao nível do transporte é tratada no MySQL através de restrições de unicidade, assegurando processamento único.



Nas próximas quatro secções devem, na descrição, resumidamente descrever num texto escoreito e legível, a forma como foi implementada. Têm de ficar muito explicitamente indicado a o que não resultou da especificação inicial (cor preta), o que foi aproveitado da especificação que receberam de outro grupo cor azul) e o que resultou de ideias posteriores vossas (cor verde). Na justificação sejam muito objetivos a explicar a razão de terem alterado a especificação inicial

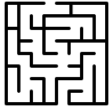
Por exemplo (não necessariamente correto), para deteção de valores anómalos:

1 Descrição

Tratámos dados anómalos resultantes de formatos de datas inválidos (por exemplo, meses com mais de 31 dias), e também considerámos sons negativos como anómalos. Os dados anómalos foram assinalados na coleção como anómalos em vez de serem apagados

2 Justificação das alterações caso tenha havido

Tínhamo-nos esquecido dos valores negativos do som e achamos que pode ser útil alguém querer consultar os valores anómalos posteriormente.



2.3 Tratamento de dados anómalos (valores de sensores errados)

1 Descrição

A validação de dados é realizada no backend intermédio, após a receção das mensagens MQTT e antes do armazenamento no MongoDB. Nesta fase são verificadas a estrutura da mensagem, os tipos de dados e a consistência dos valores recebidos.

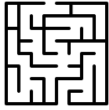
Ao contrário da especificação inicial, mensagens com timestamps inválidos não são descartadas. Em vez disso, o timestamp é convertido para *null* e o evento é marcado como inválido (*is_valid = false*), sendo ainda assim armazenado no MongoDB para garantir rastreabilidade.

A validação mantém-se maioritariamente no backend, não tendo sido adotada a divisão em dois níveis proposta pelo outro grupo. Assim, apenas dados já processados e classificados como válidos ou inválidos são persistidos, sendo posteriormente migrados para o MySQL.

2 Justificação das alterações caso tenha havido

A decisão de não descartar eventos com timestamps inválidos deve-se à necessidade de manter registo de todas as leituras recebidas, mesmo quando apresentam erros, permitindo análise.

Relativamente à abordagem do outro grupo, optou-se por não distribuir a validação entre backend e MySQL para evitar maior complexidade e dependência da lógica relacional. A centralização da validação no backend permite um controlo mais simples, previsível e consistente do fluxo de dados.



2.4 Tratamento de outliers de ruído

1 Descrição

A deteção de outliers é realizada no backend, com base na média acumulada das leituras válidas ao longo da simulação. Para cada nova leitura, calcula-se a diferença absoluta entre o valor atual e a média, sendo classificada como outlier se ultrapassar um limiar definido.

As leituras consideradas válidas são utilizadas para atualizar a média, enquanto valores classificados como outliers não influenciam esse cálculo nem o comportamento do sistema.

Ao contrário da especificação inicial, os outliers não são descartados. Em vez disso, são armazenados no MongoDB com o campo `*is_valid = false*`, garantindo que fiquem registados para análise posterior.

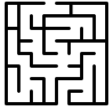
Da especificação do outro grupo foi aproveitada apenas a ideia de manter o registo dos outliers, em vez de os eliminar completamente do sistema.

2 Justificação das alterações caso tenha havido

A decisão de guardar os outliers deve-se à necessidade de manter rastreabilidade dos dados recebidos, permitindo auditoria e análise de comportamentos anómalos dos sensores.

A abordagem do outro grupo, baseada em análise de tendência com histórico no MySQL, não foi adotada por introduzir maior complexidade e dependência de lógica no nível da base de dados.

A utilização da média acumulada no backend é mais simples, suficiente para o contexto do sistema e permite uma deteção eficiente sem necessidade de histórico adicional ou processamento relacional.



2.5 Tratamento de outliers de temperatura

1 Descrição

A deteção de outliers é realizada no backend, com base na média acumulada das leituras válidas ao longo da simulação. Para cada nova leitura, calcula-se a diferença absoluta entre o valor atual e a média, sendo classificada como outlier se ultrapassar um limiar definido.

As leituras consideradas válidas são utilizadas para atualizar a média, enquanto valores classificados como outliers não influenciam esse cálculo nem o comportamento do sistema.

Ao contrário da especificação inicial, os outliers não são descartados. Em vez disso, são armazenados no MongoDB com o campo `*is_valid = false*`, garantindo que fiquem registados para análise posterior.

Da especificação do outro grupo foi aproveitada apenas a ideia de manter o registo dos outliers, em vez de os eliminar completamente do sistema.

2 Justificação das alterações caso tenha havido

A decisão de guardar os outliers deve-se à necessidade de manter rastreabilidade dos dados recebidos, permitindo auditoria e análise de comportamentos anómalos dos sensores.

A abordagem do outro grupo, baseada em análise de tendência com histórico no MySQL, não foi adotada por introduzir maior complexidade e dependência de lógica no nível da base de dados.

A utilização da média acumulada no backend é mais simples, suficiente para o contexto do sistema e permite uma deteção eficiente sem necessidade de histórico adicional ou processamento relacional.





2.6 Tratamento de Alertas de Som

1 Descrição

O fecho gradual dos corredores, despoletado pela geração de alertas de ruído foi descartado.

Em vez disso, o fecho dos corredores é agora total, isto é, após a geração de um alerta, todos os corredores são bloqueados.

2 Justificação das alterações caso tenha havido

O servidor não suporta o fecho gradual de corredores, isto é, só diminui o ruído quando todos os corredores estão bloqueados, logo, a única forma de reduzir o ruído é fechando todos os corredores de uma vez.



2.7 Tratamento de Alertas de Temperatura

1 Descrição

2 Justificação das alterações caso tenha havido



2.8 Tratamento de número de marsamis numa sala (obter pontuação)

1 Descrição

2 Justificação das alterações caso tenha havido



2.9 Implementação de Stored Procedures SQL de apoio à migração e tratamento de dados

É nesta tabela que deverão ser listados os SP que implementam mecanismos anteriores. Na quarta coluna têm de colocar um dos seguintes símbolos:

= - igual ao especificado

A(og) – Alterado com base em ideia de outro grupo

A(ni) – Alterado com base em novas ideias

N(og) – Novo com base em ideia de outro grupo

N(ni) – Novo com base em ideias novas

A – Alterado com base em novas ideias

Nome SP	Argumentos	Descrição	
...	...	...	

Texto justificativo da opção final. O que for idêntico à inicial não preencher



2.10 Implementação de Triggers

É nesta tabela que deverão ser listados os triggers. Na sexta coluna têm de colocar um dos seguintes símbolos:

= - igual ao especificado

A(og) – Alterado com base em ideia de outro grupo

A(ni) – Alterado com base em novas ideias

N(og) – Novo com base em ideia de outro grupo

N(ni) – Novo com base em ideias novas

A – Alterado com base em novas ideias

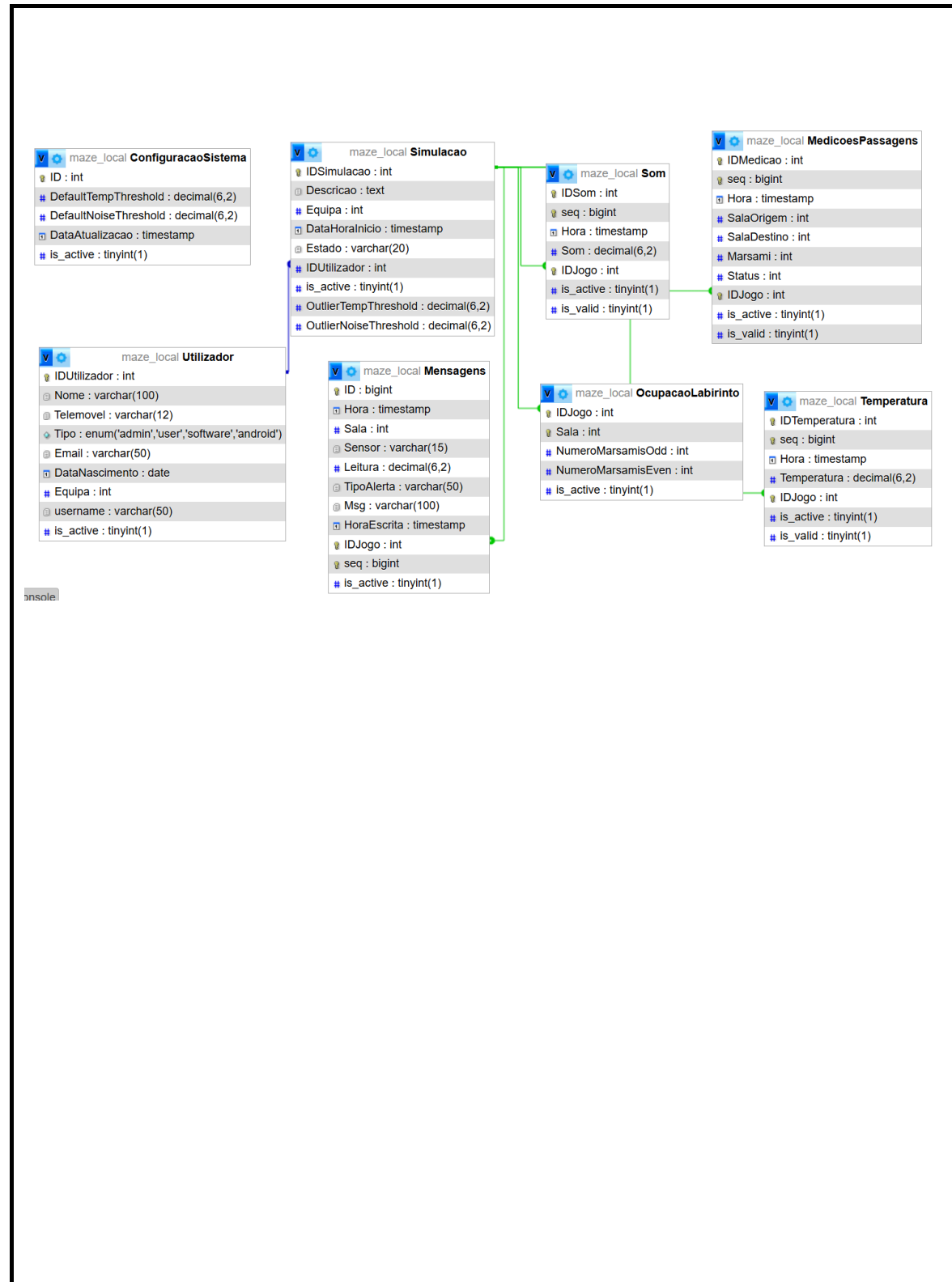
Nome Trigger	Tabela	Tipo de Operação (I, U, D)	Evento (After, Before)	Notas	

Texto justificativo da opção final. O que for idêntico à inicial não preencher



2.11 Modelo Relacional

Diagrama relacional completo implementado. Assinalar a azul alterações derivadas de outro grupo e a verde alterações novas.





Texto justificativo da opção final. O que for idêntico à não preencher

2.12 Utilizadores Base de Dados Mysql

Nesta secção deverão ser indicados os utilizadores e perfis implementados (têm de consta todos os SP usados). Assinalar a azul alterações derivadas de outro grupo e a verde alterações novas.

Tabela	Tipo de Utilizador		
	Administrador	Investigador	...
T1	U/D	–	
T2	L	I	
...			
Stored Proc.			
SP1	X	–	
...			

Em que U=Update, I Insert, D- Delete, L=Leitura, X=Executar SP e - sem permissões.

Texto justificativo da opção final. O que for idêntico à inicial não preencher



2.13 Procedimentos Manutenção da Aplicação

É nesta tabela que deverão ser listados os SP que implementam mecanismos anteriores. Na quarta coluna têm de colocar um dos seguintes símbolos:

= - igual ao especificado

A(og) - Alterado com base em ideia de outro grupo

A(ni) - Alterado com base em novas ideias

N(og) - Novo com base em ideia de outro grupo

N(ni) - Novo com base em ideia novas

A - Alterado com base em novas ideias

Nome SP	Argumentos	Descrição	
...	...	...	

Texto justificativo da opção final. O que for idêntico à inicial não preencher



2.14 Eventos de suporte à aplicação (caso existam)

É nesta tabela que eles deverão ser listados todos os eventos relevantes para o processo de migração Na quarta coluna têm de colocar um dos seguintes símbolos:

= - igual ao especificado

A(og) - Alterado com base em ideia de outro grupo

A(ni) - Alterado com base em novas ideias

N(og) - Novo com base em ideia de outro grupo

N(ni) - Novo com base em ideias novas

A - Alterado com base em novas ideias

Nome Evento	Local Execução (Mysql/Windows)	Muito breve descrição

Texto justificativo da opção final. O que for idêntico à não preencher



2.15 PrintScreen dos formulários HTML implementados



2.16 PrintScreen dos formulários Android com dados



Anexo Código SQL



Código de Triggers implementados

1. Nome Trigger: _____
// Breve Descrição
Código

2. Nome Trigger: _____
// Breve Descrição
Código

3. Nome Trigger: _____
// Breve Descrição
Código



Código Stored Procedures implementados

```
1. Nome SP: CriarUtilizador
// Breve Descrição
Código

2. Nome SP: EditarExperiência
// Breve Descrição
Código
```